# Ders 12 Çalışma Özeti

## Ders 12 Çalışma Özeti

### Genel Konular

- Bu kayıt bir uygulama (lab) oturumudur. Bir araştırma görevlisi Quiz 1 ve Ödev 1'in çözüm yaklaşımını anlattı.
- Renk matrisinin histogramı: farklı renkleri ve sayılarını bulma algoritması.
- Dinamik bellek: `malloc`, `calloc` ve `realloc` farkları.
- Dizi ile bağlı liste (linked list) karşılaştırması: bellek kullanımı ve rastgele erişim (random access).
- Kodlama alışkanlıkları: anlamlı değişken adları, `free` kullanımı, global değişkenlerden kaçınma.
- Global değişkenlerin çok iş parçacıklı (multi-thread) programlarda yarış durumu (race condition) üretmesi.
- Yığın (stack) ile kodlanmış bir kelimeyi çözme ve iki kelimeyi karşılaştırma.

### Hocanın Özellikle Vurguladığı Kısımlar

- Bir resmin bütün piksellerini gezmek zorunludur. Bu adımın karmaşıklığı $O(n \cdot m)$'dir ve bundan kaçış yoktur.
- Farklı renkler dizisinde arama, renk bulununca hemen durdurulmalıdır. Dizinin sonuna kadar gitmeye gerek yoktur.
- Bellek ayırma (allocation) mümkün olduğunca dinamik yapılmalıdır. Statik boyutlar yalnızca kendi testleriniz için kullanılmalıdır.
- Ayrılan her bellek kullanım bitince `free` ile geri verilmelidir. C ve C++'ta Java ve Python'daki gibi çöp toplayıcı (garbage collector) yoktur.
- Global değişken kullanılmamalıdır. Aynı iş fonksiyona parametre göndererek yapılabilir.
- Değişken adları ne tuttuğunu anlatmalıdır. `maxColorRange` iyi bir addır. `int a = 256;` kötü bir addır.
- Basit dizi ve string işlemleri (karşılaştırma, alt string alma) hazır fonksiyonla değil, kendi kodunuzla yapılmalıdır. `malloc` ve `printf` gibi sistem fonksiyonları bu kuralın dışındadır.
- Veri yapıları dersinde asıl amaç doğru yapıyı seçmek ve uygun kullanmaktır.

### Kısa Tekrar Notları

- `malloc(boyut)`: tek parametre alır. Belirtilen bayt kadar yer ayırır. İçeriği sıfırlamaz.
- `calloc(eleman_sayısı, eleman_boyutu)`: iki parametre alır. Ayırdığı alanı sıfırla doldurur.
- `realloc(işaretçi, yeni_boyut)`: daha önce ayrılmış alanı genişletir veya küçültür.
- Her `realloc` çağrısı arka planda iş yapar (sistem çağrısı, parçaları birleştirme). Bellek kazandırır ama CPU ve zaman harcar.
- Bağlı listede her eleman ek bir adres tutar. Diziye göre eleman başına daha çok yer gerekir.
- Bağlı listede doğrudan erişim yoktur. 5. elemana ulaşmak için baştan itibaren tek tek gezilir.
- Dizide `uniques[5]` gibi doğrudan erişim vardır.
- `printf("%3d", x)` ile sayılar hizalı yazdırılır.
- Yapı (struct) tanımı bir değişken değil, bir tiptir. Dosya başında tanımlanır. Liste başı (`head`) işaretçisi ise `main` içinde tutulabilir.

- Kodlanmış kelimede harfler yığına itilir (push). Bir rakam gelince o kadar harf yığından çekilir (pop).

## Detaylı Açıklamalar

### Quiz 1: Renk Matrisinin Histogramı

- Girdi: $N \times M$ boyutlu bir renk matrisi. Renk değerleri 0 ile 255 arasındadır.
- Varsayım: farklı renk sayısı en fazla $k = 256/3$ kadardır.
- Amaç: matristeki farklı renkleri ve her birinin kaç kez geçtiğini bulmak.
- Çözümde iki dizi kullanıldı:
  - `uniques`: bulunan farklı renkler.
  - `counts`: her rengin sayısı. İndisler `uniques` ile ortaktır.
- Bu iki dizi tek bir iki boyutlu dizide de tutulabilir. 0. sütunda renk, 1. sütunda sayı olur.
- $N$ ve $M$ makro ile tanımlandı. Kod içinde birden fazla yerde kullanılan değerleri yönetmek kolaylaşır.

Çözüm kodunun ana kısmı (ekrandan, 00:09:30):

```
int maxColorRange = 256; // 0-255

int i, j, t; // indisler
int k = (int)maxColorRange / 3; // varsayima gore kabul edilen "k/3" farkli renk sayisi
int found; // histogram icin kullanilan flag.
int colorCount;

int* uniques = (int*)malloc(k * sizeof(int));
int* counts = (int*)calloc(k, sizeof(int));

colorCount = 0;
for (i = 0; i < N; i++)
{
    for (j = 0; j < M; j++)
    {
        found = 0;
        t = 0;
        while ((found == 0) && t < colorCount) {
            if (uniques[t] == test[i][j]) {
                found = 1;
                counts[t] = counts[t] + 1;
            }
            t++;
        }
        if (found != 1) {
            uniques[colorCount] = test[i][j];
            counts[colorCount] = 1;
            colorCount++;
        }
    }
}
```

- `while` döngüsü iki nedenle biter:
  - Renk bulunmuştur (`found == 1`). O rengin sayacı artırılır.
  - Bütün farklı renkler gezilmiş, renk bulunamamıştır.
- Renk bulunamadıysa yeni bir renktir. `uniques` dizisine eklenir, sayacı 1'den başlar. `colorCount` bir artar.
- Kodda `malloc` ve `calloc` örnek olsun diye birlikte kullanıldı.

### Daha İyi Bellek Kullanımı

- Soruda gömülü sistem (embedded system) senaryosu vardı. Bu yüzden bellek kullanımı önemliydi.
- Gösterilen çözüm baştan $k$ kadar yer ayırır. Renk sayısı azsa bu alanın bir kısmı boş kalır.
- **Alternatif 1: `realloc`**
  - Her yeni renk geldiğinde dizi bir eleman büyütülür.
  - Bellek daha verimli kullanılır.
  - Ama her `realloc` CPU ve zaman harcar. Gömülü sistemde bu da bir maliyettir.
- **Alternatif 2: bağlı liste**
  - Her yeni renk için bir düğüm oluşturulur ve listenin sonuna bağlanır.
  - Kullanılmayan yer ayrılmaz. Yönetimi `realloc`'tan kolaydır.
  - Dezavantaj 1: her düğüm ek bir adres tutar.
  - Dezavantaj 2: rastgele erişim yoktur. Bir elemana ulaşmak için baştan gezmek gerekir.
- Bir öğrenci dizilerin sabit boyutla tanımlanmasının farkını sordu.
  - Asistan, gerçek programda en büyük renk değerinin kullanıcıdan okunacağını söyledi.
  - Bu durumda boyut önceden bilinmez. `malloc` veya `calloc` kullanmak zorunludur.

### Global Değişkenlerin Sakıncası

- İki iş parçacığı (thread) aynı global değişkene erişsin. Biri değeri 1, diğeri 0 yapsın.
- Programcı önce X'in, sonra Y'nin çalışmasını bekler.
- Ama işlemcideki kesintili zamanlayıcı (preemptive scheduler) programcının kontrolünde değildir. Y üst üste birkaç kez çalışabilir.
- Sonuçta global değişken beklenmeyen bir değer tutabilir. Bu hatayı bulmak çok zordur.
- Kritik bölge (critical section) ve mutex konuları işletim sistemleri dersinde ayrıntılı işlenecektir.
- Global değişkenle yapılan her şey yerel değişken ve parametre aktarımıyla yapılabilir. Global değişken bir zorunluluk değil, bir tercihtir.

### Ödev 1: Yığın ile Kodlanmış Kelime

Örnek (ekrandan, 00:28:30): `"Veri"` ile `"Vezir3 Rize2"` eşit mi?

- Harfler okunduğu sırayla yığına itilir. Bir rakam gelince o sayı kadar pop yapılır.
- Adımlar:

| Girdi | İşlem | Yığın (alttan üste) |
|---|---|---|
| `V e z i r` | 5 push | `V e z i r` |
| `3` | 3 pop (`r`, `i`, `z`) | `V e` |
| `R i z e` | 4 push | `V e R i z e` |
| `2` | 2 pop (`e`, `z`) | `V e R i` |

- Sonuçta yığında "VeRi" kalır. Tahtadaki örnekte büyük/küçük harf ayrımı yapılmadı. Kelime "Veri" olarak kabul edildi.
- Silme işlemi sondan başa doğru yapılır. Bu push/pop mekanizmasının yığın sorusu olduğunu gösterir.
- **Karşılaştırma için yığın yaklaşımı:**
  - Orijinal kelime de ayrı bir yığına itilir.
  - İki yığından aynı anda birer karakter çekilir ve `if` ile karşılaştırılır.
  - C'de tek karakter == ile karşılaştırılabilir. Ama string doğrudan == ile karşılaştırılamaz.
  - Bütün karakterler eşitse ve iki yığın aynı anda boşalırsa kelimeler eşittir.
- Asistan hazır `strcmp` yerine bu yaklaşımı tavsiye etti.
- Stack dışındaki çözümler de kabul edildi. Ama beklenen, sorunun yığın sorusu olduğunu görmekti.

**Ders sonrası notlar**

- Asistan `malloc` parametresinin “bit sayısı” olduğunu söyledi. Doğrusu: `malloc` parametresi **bayt** sayısıdır.
- Asistan `strcmp`’nin iki veya üç parametre alabildiğini söyledi. Doğrusu: `strcmp` iki parametre alır. İlk $n$ karakteri karşılaştıran üç parametreli fonksiyon `strncmp`’dir.
- Asistan adres için `int` yerine `long` gibi daha büyük bir tamsayı tipi gerekebileceğini söyledi. C’de adres bir işaretçi (pointer) tipinde tutulur. 64 bit sistemlerde işaretçi genellikle 8 bayt, `int` genellikle 4 bayttır.
- Yukarıdaki kod parçasında `N`, `M` ve `test` matrisi kodun ekranda görünmeyen kısmında tanımlıdır. Parça kod olduğu için derlenmedi.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.